

StrataCL: Fabric-Native Communication Library for Production Supernodes

Tiancheng Hu^{1,6}, Jin Qin^{2,3}, Yuzheng Wang^{1,6}, Ke Liu^{2,3}, TangShengsheng Li⁴, Sheng Wang⁴, Zhongzhe Hu⁴, Tianlun Hu⁴, Wei Wang⁴, Lijun Li⁴, Jingbin Zhou⁴, Xiaoming Bao⁴, Hongwei Sun⁴, Jieru Zhao⁵, Huimin Cui^{2,3}, Tao Xie^{1,6}, Chenxi Wang^{2,3}

¹Peking University ²SKLP, Institute of Computing Technology, CAS

³University of Chinese Academy of Sciences ⁴Huawei Technologies Company Ltd.

⁵Shanghai Jiao Tong University

⁶Beijing Tongming Lake Information Technology Application Innovation Center, China

Abstract

Modern distributed AI workloads run across hundreds of accelerators, making communication a major bottleneck. Existing communication libraries remain largely buffer-centric because user and communication buffers are managed separately, causing redundant data copies or costly user-buffer registration. This paper presents StrataCL, a zero-redundancy and fabric-native communication library for production supernodes. StrataCL introduces registration-on-allocation to realize user-buffer direct communication, and designs communication operators with workload-balanced NPU-core partitioning and NPU-driven SDMA offloading to exploit supernode architecture features. On the Huawei CloudMatrix384, StrataCL improves collective bus bandwidth by up to 1.6× and improves MoE dispatch/combine bus bandwidth by up to 1.4×. Across three production workloads, StrataCL improves LLM inference throughput by 1.9×, reduces P99 TTFT by 2.2×, and reduces LLM and Recsys training iteration time by 1.4× and 1.3×, respectively.

1 Introduction

Modern large-scale production applications increasingly rely on distributed execution across hundreds of accelerators, making inter-device communication a major performance bottleneck [6, 82]. For example, communication operators account for approximately 10%–40% of end-to-end time in distributed large language model (LLM) inference [14, 47] and 30%–45% in training [31, 55]. As model size and cluster scale continue to grow, the communication-to-computation ratio further increases and may exceed 50% when compute capability scales faster than interconnect bandwidth [54].

Common communication libraries, such as NCCL [42], RCCL [4], and HCCL [19], are the backbone of large-scale training and serving on GPU [3, 40] and NPU [36] clusters. However, their conventional paths are largely *buffer-centric* [65]. User buffers are allocated by application frameworks (e.g., PyTorch [51], SGLang [79]), while communication buffers are separately managed by the communication

runtime. This allocator-level separation prevents the communication runtime from directly operating on user buffers, as they are not registered as remotely accessible. As a result, data incurs redundant staged copies through internal communication buffers, increasing latency and consuming extra HBM, as demonstrated by previous work [33, 65]. Recent libraries mitigate this overhead through user-buffer direct communication, such as NCCL User Buffer Registration [48] and HCCL-zero-copy [22].

On traditional clusters built over scale-out RDMA fabrics, e.g., InfiniBand [27] and RoCE [26], redundant data movement remains difficult to eliminate, and deploying user-buffer registration is challenging. RDMA-based memory registration is costly [16, 71, 72], which needs to pin the GPU pages, establish DMA mappings through PCIe BAR, create meta-data such as memory regions, and exchange the information across ranks. These steps introduce non-negligible overhead, which can reach several milliseconds [49] and may offset the benefit of user-buffer direct communication, especially under dynamic allocation patterns such as MoE serving.

Opportunities in supernode architectures. Supernodes, such as GB200 NVL72 [41] and CloudMatrix384 [83], are emerging server architectures that tightly integrate hundreds of accelerators through scale-up fabrics, such as NVLink [45] and UB [35]. This paper focuses on Huawei CloudMatrix384 (CM384) [74, 83] as a representative platform. CM384 is built around the Unified Bus (UB), a scale-up fabric that connects 384 Ascend NPUs within a single supernode (architecture details in §2.1). The following discussion highlights the opportunities that such supernode architectures create for communication-library design.

High-bandwidth and low-latency inter-NPU communication. UB provides nearly 400 GB/s bandwidth and nanosecond-scale remote HBM access latency, substantially narrowing the gap between remote and local HBM access. Within a supernode, UB data transfer is fast enough that copies between user buffers and internal communication buffers can account for a significant fraction of communication latency, making user-buffer direct communication especially beneficial (§2.2).

Global unified physical address space. CM384 maps the HBM physical address ranges of all NPUs into a unified UB address space. A memory region can therefore become remotely accessible by mapping a local virtual address to the corresponding remote NPU HBM physical address, which fundamentally simplifies remote-access memory registration and makes it much more lightweight (§2.3).

Challenges. Despite these opportunities, fully exploiting CM384’s architectural features for communication optimization faces two critical challenges:

How to remove user-buffer registration overhead from the communication critical path. Although the global unified physical address space makes remote-access memory registration much more lightweight (*i.e.*, microsecond-scale), just-in-time registration still adds non-negligible latency to communication, especially in large-scale scenarios due to poor scalability. A naive solution is to pre-register a sufficiently large memory pool and require the application framework to allocate user buffers from this pool. While this removes the registration overhead from the critical path, it leads to severe memory waste and intrusive framework modifications (§3.1).

How to design fabric-native communication operators for supernodes. With UB’s high bandwidth and low latency, data transfer latency becomes less dominant in communication, while synchronization overhead becomes increasingly visible, especially for small payloads. Moreover, UB’s hierarchical topology introduces non-uniform memory access. Under topology-unaware workload partitioning, NPU cores assigned to slower or heavier peer transfers may finish much later than others, creating a long-tail problem. Finally, saturating UB bandwidth requires many NPU cores to issue remote-memory operations concurrently, which can contend with overlapped compute kernels and reduce the benefit of compute–communication overlap (§3.2).

StrataCL. This paper proposes StrataCL, a zero-redundancy and fabric-native communication library for production supernodes. StrataCL eliminates redundant data copies along the communication path and designs communication operators considering supernode architecture features, directly addressing the two challenges above.

For the first challenge, StrataCL proposes *registration-on-allocation*, based on the key insight that a long interval typically separates a buffer’s physical memory allocation from its first use in communication. For each physical allocation, StrataCL intercepts memory APIs and asynchronously registers the region in the background, deferring only a lightweight completion check until a communication operator first accesses it. StrataCL further uses shadow virtual addressing, assigning each NPU a disjoint virtual-address range and mirroring allocated buffers at the same virtual address on peer NPUs. This allows communication operators to issue remote access without peer-to-peer address translation. To support VMM API-based allocators, StrataCL incrementally

updates shadow mappings for remapped regions and uses a fast broadcast path to exchange registration metadata (§5).

For the second challenge, StrataCL builds an efficient *full-mesh* programming abstraction to avoid the frequent synchronization overhead of multi-step algorithms using remote load/store instructions. To address core-level long tails caused by non-uniform memory access of UB, StrataCL proposes a *workload-balanced NPU-core partitioning* to avoid the long-tail problem. Finally, under compute–communication overlap, saturating UB bandwidth requires many NPU cores to concurrently issue remote-memory access instructions, which can contend with overlapped compute kernels. StrataCL addresses this with NPU-driven SDMA offloading, where NPU cores only submit DMA descriptors and the SDMA engine asynchronously performs data movement, quickly releasing NPU cores back to overlapped computation (§6).

StrataCL is evaluated on a CM384 supernode using up to 512 NPU dies and three real-world production applications. The results show that StrataCL improves both operator-level communication efficiency and end-to-end application performance. In microbenchmarks, StrataCL achieves up to 1.6× higher bus bandwidth for collective communication and up to 1.4× higher bus bandwidth for MoE dispatch/combine compared with state-of-the-art production communication libraries. In end-to-end evaluation, StrataCL improves LLM inference throughput by 1.9× and reduces P99 TTFT by 2.2×. For training workloads, StrataCL accelerates LLM training and recommendation-model training by up to 1.4× and 1.3×.

This paper makes the following key contributions:

- It proposes *registration-on-allocation*, enabling transparent user-buffer direct communication while keeping registration off the critical path, remaining compatible with VMM API-based allocators, and demonstrating portability on NVIDIA GPUs.
- It designs fabric-native communication operators for supernodes, combining a full-mesh programming abstraction, workload-balanced NPU-core partitioning, and NPU-driven SDMA offloading to reduce synchronization overhead, core-level long tails, and NPU-core contention under compute–communication overlap.
- It integrates StrataCL into production frameworks and demonstrates significant end-to-end speedups. The source code will be released upon publication.

2 Background and Observations

2.1 Supernode Architectures

CM384 integrates 384 Ascend 910C NPUs [36] into a supernode interconnected by UB. As shown in Figure 1, a CM384 contains 12 compute racks, each with four nodes and eight NPUs per node. CM384 uses a two-level UB switching hierarchy in which L1 UB switches connect the eight NPUs within a node, while L2 UB switches aggregate L1 switches across nodes for inter-node connectivity.

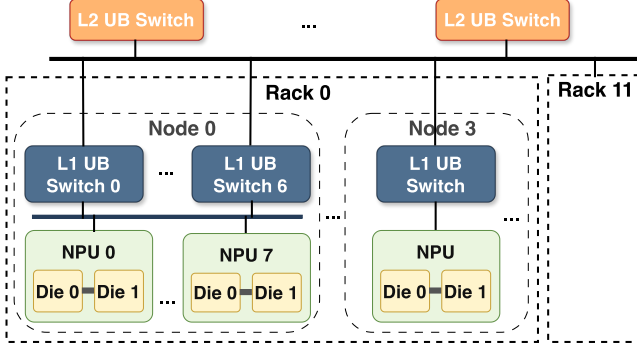


Figure 1. Huawei CloudMatrix384 supernode topology.

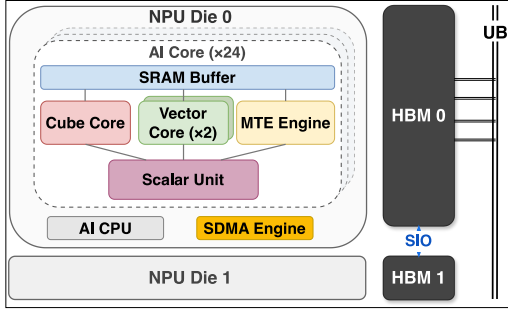


Figure 2. Simplified architecture of an Ascend 910C NPU. Each NPU contains two dies, each with 24 AI Cores; each AI Core integrates one Cube Core and two Vector Cores.

Each Ascend 910C NPU is a dual-die package connected by a high-bandwidth SIO fabric with up to 540 GB/s aggregate bandwidth. As shown in Figure 2, each die contains 24 AI Cores. Each AI Core integrates one Cube Unit (AIC) for matrix computation and two Vector Cores (AIV) for vector processing, coordinated by a scalar unit. Since communication kernels mainly run on AIV cores, this paper uses *NPU core* to refer to an AIV core. Each die has 64 GB HBM with 1.6 TB/s bandwidth. Each AI core includes a KB-level unified buffer and Memory Transfer Engines (MTEs) for data movement between HBM and on-chip SRAM buffers. An SDMA engine further supports inter-NPU data movement, typically launched through host APIs.

2.2 User-Buffer Direct Communication

Compared with RDMA, which typically provides only tens of GB/s of effective bandwidth, UB offers substantially higher bandwidth and narrows the gap between inter-NPU communication and local HBM access. This makes user-buffer direct communication especially beneficial on supernodes. Figure 3a reports the AllGather bus bandwidth [50] of HCCL and HCCL-zero-copy at 32 ranks. HCCL-zero-copy improves bus bandwidth by more than 30% for all payloads above 8 MiB by eliminating redundant data movement on the staging path. For an AllGather across N ranks with M bytes per rank, HCCL copies M bytes from each user input buffer into

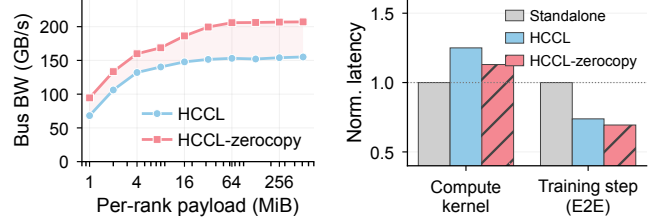


Figure 3. (a) Bus bandwidth of HCCL vs. HCCL-zero-copy AllGather at 32 ranks. (b) Normalized compute-kernel and training step latency in overlap scenarios.

an internal communication buffer and then copies the gathered $N \cdot M$ bytes back to user output buffers, adding $(1+N) \cdot M$ bytes of staging traffic. HCCL-zero-copy bypasses these end-point copies by communicating directly over user buffers. Although chunked-pipelined execution can overlap part of the staging cost with network transfer, it cannot fully hide the extra copy overhead, and the pipeline still occupies NPU cores and contends with concurrent compute kernels under compute-communication overlap [33, 65].

User-buffer direct communication also reduces bandwidth interference. In overlap-heavy training pipelines such as Fully Sharded Data Parallel (FSDP) [77], parameter AllGather prefetches run concurrently with compute kernels, while staged copies consume HBM bandwidth that would otherwise be available to computation. Figure 3b quantifies this effect in a DeepSeek V3.2 671B [11] training deployment on 512 NPU dies, with detailed settings in §8.2. HCCL slows the concurrent compute kernel by 25%, whereas HCCL-zero-copy reduces the slowdown to 13%. This lower interference directly improves training performance. Compared with a no-overlap serial baseline, overlapped execution reduces training step time by 26% with HCCL and by 31% with HCCL-zero-copy, yielding an additional 6% end-to-end speedup from eliminating staging-induced contention.

2.3 Inter-NPU Memory Mapping

The *global unified physical address space* of UB makes inter-NPU memory access structurally simpler than on RDMA. Accessing a remote buffer over RDMA requires the target endpoint to register the buffer and send the remote key and remote virtual address to peer RNICs. In contrast, UB exposes remote HBM through a unified physical address space. A local virtual address can be mapped to remote HBM physical pages and then accessed like local HBM. Figure 4a reports memory registration latency under three payload sizes. UB registration is consistently an order of magnitude faster than RDMA registration, achieving a 9× speedup on average.

UB registration consists of four logical steps. The remote rank exports a physical-memory handle, and the local rank imports the handle, reserves a local virtual address, and maps the reserved VA to the remote rank’s HBM. Figure 4b shows

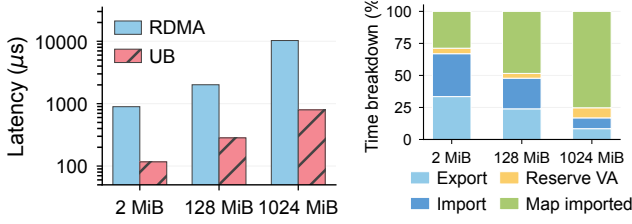


Figure 4. Inter-NPU memory registration on supernode. (a) Registration latency comparison of RDMA and UB. (b) Time breakdown of UB registration, normalized to the total.

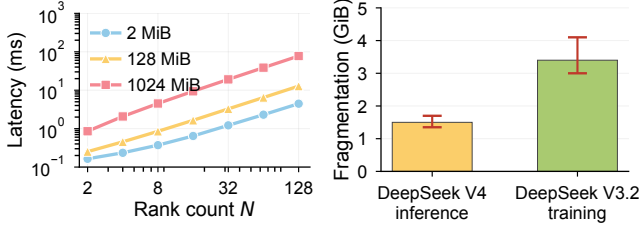


Figure 5. (a) User-buffer registration overhead varying with the number of ranks. (b) HBM fragmentation on CM384 when PyTorch’s expandable-segment allocator is disabled.

the latency breakdown of these steps. For smaller payloads, handle export and import can cause significant delays. As the payload grows, the mapping step becomes dominant because NPU page-table updates scale with the registered size.

3 Motivation

3.1 User-buffer Registration

Costly and poorly scalable just-in-time registration. Although UB reduces inter-NPU memory registration to the microsecond scale, this overhead remains non-negligible compared with communication operators, whose latency typically ranges from hundreds of microseconds to tens of milliseconds. More importantly, just-in-time registration scales poorly with rank count since each mapping operation acquires a write-side page-table lock, and per-peer mappings on the same rank must be serialized. As a result, registering a full N -rank communication group incurs approximately $O(N)$ latency per rank and grows nearly linearly with rank count, as shown in Figure 5a. Caching registered buffers can amortize this cost for static and repetitive communication patterns, but it is much less effective under dynamic memory allocation, where tensor shapes change across invocations. A representative example is MoE inference, whose dispatch buffer sizes vary across batches due to dynamic token routing [9]. In such cases, just-in-time registration repeatedly places the registration overhead on the communication critical path, weakening or even offsetting the benefit of user-buffer direct communication.

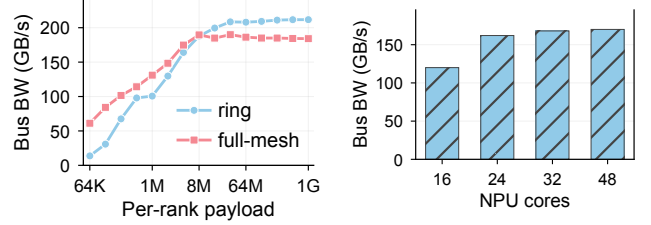


Figure 6. (a) Bus bandwidth of ring vs. full-mesh AllGather. (b) Achievable UB bus bandwidth of current NPU cores.

Table 1. Access latency and single-direction bandwidth of two NPUs across different access relative locations.

Relative location	Latency (μ s)	Bandwidth (GB/s)
Die-to-Die	0.2	210
Intra-node	0.7	170
Inter-node	2.1	150

Pool-based approaches fail to achieve both HBM efficiency and framework compatibility. A simple workaround is to pre-register a communication memory pool [33], which removes registration from the critical path but turns registration overhead into an HBM capacity tax. Since this pool is dedicated to communication buffers, its capacity is isolated from the application framework allocator, reducing the maximum supported batch size. Some approaches [65] allow the framework and communication library to share a registered pool, but they rely on static virtual-to-physical mappings, which conflict with modern on-demand VMM API-based memory management such as PyTorch’s expandable-segment allocator [60]. Figure 5b quantifies the memory fragmentation¹ on CM384 when expandable segments are disabled. DeepSeek V4 Flash [12] inference loses 1–2 GiB of usable memory per NPU die, while DeepSeek V3.2 671B training loses 3–4 GiB, which directly reduces the supported batch size and leads to end-to-end throughput degradation.

3.2 Communication Operator Execution

Synchronization overhead becomes a critical bottleneck. Algorithms such as ring [52], PAT [29], and recursive halving-doubling [67] are effective in RDMA scale-out clusters because they reduce instantaneous fanout, mitigate network contention, and avoid excessive traffic on slow cross-node paths. However, these algorithms rely on multiple interdependent communication steps, whose cumulative synchronization overhead becomes a critical bottleneck in supernode fabrics, especially for small-to-medium messages. In contrast, full-mesh communication completes in a single logical step by issuing concurrent memory instructions. As shown in

¹Memory fragmentation here refers to the sum of the available memory blocks in PyTorch under the current maximum supported batch size.

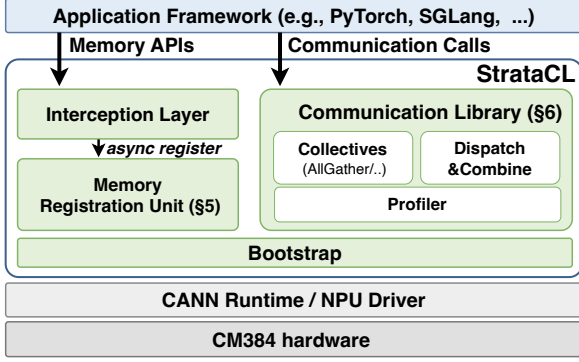


Figure 7. StrataCL design overview.

Figure 6a, full-mesh outperforms ring across the small-to-medium range, on average over $2\times$ faster below 1 MiB and up to $4.5\times$ at 64 KiB, and remains ahead through 8 MiB. The latency breakdown shows that synchronization accounts for more than 50% of the ring’s end-to-end time at small payloads and reaches $\sim 77\%$ at 64 KiB. Ring becomes preferable beyond 16 MiB, when lower cross-node traffic and fanout contention outweigh the synchronization cost.

Non-Uniform Memory Access (NUMA) creates core-level long tails. Although the supernode exposes a unified global address space, its two-level switching topology makes memory-access performance depend on the relative location of source and destination NPUs. As shown in Table 1, die-to-die access within the same NPU reaches 210 GB/s at 0.2 μ s, while intra-node remote HBM access drops to 170 GB/s at 0.7 μ s, and inter-node access further drops to 150 GB/s at 2.1 μ s. Compared with die-to-die access, inter-node access has roughly $10\times$ higher latency and 29% lower bandwidth. This non-uniform memory access cost makes NPU cores assigned to slower or heavier peer transfers finish much later than others, creating a core-level long tail problem, especially for high-concurrency full-mesh operators.

NPU-core contention limits overlap. To saturate UB bandwidth, an NPU must issue many concurrent remote-memory operations, such as remote loads/stores or peer memory copies, which requires many NPU cores to participate in data movement. As shown in Figure 6b, bus bandwidth reaches 95% of peak only when 24 NPU cores are used, consuming half of the 48 NPU cores on each NPU die. This substantially reduces the NPU-core budget available for concurrent computation kernels and can cause severe NPU-core contention in compute-communication overlap scenarios, such as FSDP parameter prefetch [77] in LLM training and Two-micro-Batch Overlapping [10] in LLM inference.

4 Design Overview

As shown in Figure 7, StrataCL sits between the application framework and the CM384 runtime stack and consists of

three main modules. First, the *memory registration unit* transparently makes framework-allocated user buffers remote-accessible. Through the interception layer, StrataCL hooks application memory APIs and triggers asynchronous memory registration in the background. Second, the *communication library* provides both common collective operators and workload-specific operators. It also integrates a fine-grained NPU-core profiler to identify operator bottlenecks and guide optimization. Finally, the *bootstrap* component initializes communicator groups, establishes metadata-exchange channels, and assigns each NPU a disjoint virtual-address range.

5 Memory Registration

5.1 Registration-on-allocation

The key insight behind StrataCL is that a buffer’s physical memory allocation is usually separated from its first communication use by a long interval, *e.g.*, at least 2.6 s in LLM inference (§9.4). Modern framework allocators reserve large virtual-address segments at startup and reuse them across iterations [32, 58, 79]. Therefore, common-case tensor allocation typically obtains a virtual address from pre-mapped memory segments, such as those managed by the PyTorch caching allocator [58]. StrataCL exploits this allocation-to-communication gap with an asynchronous *registration-on-allocation* mechanism. Instead of registering a buffer when it is accessed by a communication operator, StrataCL registers the underlying physical allocation for remote access in the background immediately after allocation. This keeps remote-access memory registration off the critical path while remaining transparent to the application framework.

Figure 8 illustrates the workflow. When the application on NPU 0 allocates buffer A, StrataCL makes A remotely accessible in the background without blocking the application. ① The application allocates a virtual address slot and maps it to the corresponding local HBM physical pages, following the normal allocation path. ② After the allocation returns, the memory-registration unit asynchronously exports A’s physical-memory handle and broadcasts the required metadata to peer NPUs, including the owner NPU, virtual address, size, and physical mapping information. ③ Upon receiving the metadata, each peer NPU maps the same virtual address to NPU 0’s remote HBM physical pages through UB (Phase 1). To ensure correctness, StrataCL defers a readiness check until a communication operator first touches the region (Phase 2). This check is purely local, where each NPU only verifies that its own UB mappings for the region have completed, without cross-NPU synchronization, incurring negligible overhead.

5.2 Shadow Virtual Addressing

A physical memory may be mapped by different virtual addresses on different NPUs. This forces communication operators to translate peer-buffer addresses before remote accesses

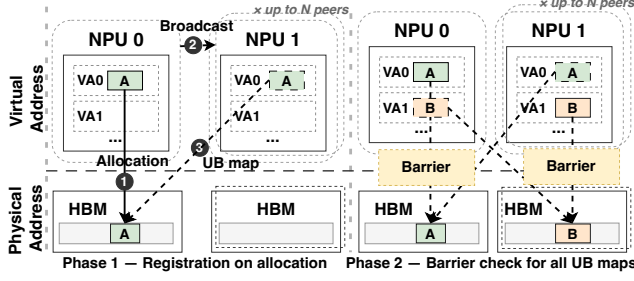


Figure 8. Registration workflow in StrataCL. When a physical allocation is intercepted, an asynchronous registration starts (Phase 1). When a communication operator first accesses the region, StrataCL performs a readiness barrier to ensure that the corresponding UB mappings have completed (Phase 2).

and requires each NPU to maintain address-translation metadata, which becomes increasingly expensive at scale.

StrataCL eliminates this overhead with *shadow virtual addressing*, which makes a buffer visible at the same virtual address on all NPUs. The key idea is to *decouple virtual-address planning from physical-memory ownership*. During initialization, StrataCL assigns each NPU a disjoint virtual-address range, so the ranges of different NPUs never overlap. Since the virtual-address space is much larger than the physical HBM capacity, reserving these disjoint ranges introduces negligible address-space pressure. When NPU i allocates a buffer at virtual address v , StrataCL performs a shadow mapping on every peer NPU. Each peer reserves the same virtual address v in its own address space and maps it to the physical HBM pages of NPU i through UB’s global unified physical address space. After this mapping, all NPUs can access the buffer using the identical virtual address v . As a result, communication operators can issue remote accesses without peer-to-peer address translation, simplifying the implementation and reducing overhead.

5.3 Integration with VMM API-based allocator

The mechanisms above assume that a buffer’s virtual-to-physical mapping remains fixed after allocation, so registration can complete before the buffer is first used. VMM API-based allocators, such as PyTorch’s expandable-segment allocator [60], break this assumption by reserving a large virtual-address range and mapping physical pages on demand to reduce fragmentation. Consequently, new physical pages may be mapped to an existing virtual address after the initial allocation and remain invisible to peer NPUs.

StrataCL supports VMM API-based allocators by treating the runtime mapping as a lightweight incremental registration. It intercepts the mapping and asynchronously re-applies the shadow mapping for the newly mapped region. This avoids registering the entire virtual address range, which would be prohibitively expensive. If a freshly mapped region

is accessed before the asynchronous UB mapping completes, StrataCL still uses a synchronization barrier to ensure correctness, which is no worse than just-in-time registration.

Runtime remapping mainly appears in workloads with dynamic allocation patterns. For example, MoE serving with continuous batching [76] can trigger remapping because both the active-token count and expert-routing distribution vary across batches [81]. To reduce the metadata exchange overhead, StrataCL uses a fast broadcast path through CPU-side direct access to peer host DRAM, as described in §7. Meanwhile, runtime remapping is triggered in less than 4% of request batches in MoE serving. Even when it occurs, a gap of tens of milliseconds is still observed between the map operation and the first communication use, typically due to intervening computation, which is sufficient to hide UB mapping latency. As a result, StrataCL supports VMM API-based allocators with less than 0.6% end-to-end overhead.

5.4 Memory Deregistration

Deallocation is handled symmetrically to allocation. Tensor frees that only return a sub-allocation block to the caching allocator do not change the underlying physical memory and trigger no action. Deregistration is required only when physical memory is actually released, such as `ac1rtFree` on a physical segment or a VMM unmap that decommits pages. The key property enabling non-blocking deregistration is that UB address translation allows multiple virtual addresses to map to the same physical page. Therefore, each peer-side shadow mapping is only an independent alias of the released region. When an NPU frees a region, StrataCL can immediately reclaim the local virtual address, while peer-side UB unmappings proceed asynchronously in the background. To avoid stale remote accesses, the released physical segment is not reused for new communication allocations until all corresponding UB mappings have been removed.

6 Communication Library

6.1 Full-Mesh Programming Abstraction

Remote-slice execution model. As shown in §3.2, synchronization overhead becomes increasingly expensive in the supernode. StrataCL therefore adopts a full-mesh execution model for small-to-medium payloads, enabled by UB remote load/store. StrataCL decomposes communication operators into a set of *remote-slice transfers*, represented as

$$\langle \text{peer}, \text{src}, \text{dst}, \text{bytes}, \text{op}, \text{flag} \rangle$$

where `peer` identifies the remote NPU, `src` and `dst` are virtual addresses in the shared address space, `bytes` specifies the slice size, `op` specifies the remote-memory action, and `flag` encodes the optional dependency signal. Different communication operators are expressed by changing two fields. The *slice map* defines which byte ranges are accessed by which

peers, while the *remote-memory action* specifies whether the transfer performs a load, a store, or an atomic instruction.

Minimal synchronization. The full-mesh execution model reduces synchronization at the algorithm level, and StrataCL further minimizes synchronization in operator execution according to operator semantics. For data-movement operators such as AllGather and AllToAll, StrataCL uses a pull mode, where each rank directly reads the remote slices and writes them into local output buffers, avoiding producer-consumer handshakes. For reduction-bearing operators such as AllReduce and ReduceScatter, remote operands are first loaded into the on-chip SRAM buffer and then accumulated into the local destination buffer. When multiple NPU cores update the same memory, StrataCL uses the atomic instruction of the MTE during local write-back, so a store from SRAM to HBM becomes an atomic read-modify-write operation, which allows multiple NPU cores to accumulate partial results into the same destination without software locks.

Unified kernel skeleton. The full-mesh abstraction separates operator semantics from backend execution. The operator frontend only generates the slice map and corresponding src/dst virtual addresses, while the backend schedules and executes remote-slice transfers. This separation allows backend optimizations to be applied below the unified programming abstraction, including workload-balanced NPU-core partitioning (§6.2) and NPU-driven SDMA offloading (§6.3).

6.2 Workload-balanced NPU-core Partitioning

A key challenge in implementing full-mesh operators with highly concurrent NPU cores is avoiding core-level long tails caused by imbalanced workload partitioning. The imbalance comes from two sources. (i) *Uneven traffic.* For routing- and shape-dependent operators, such as AllToAllv and MoE dispatch/combine, different peers may exchange different traffic volumes. (ii) *Non-uniform memory access.* UB’s hierarchical topology introduces non-uniform latency and bandwidth, so the same traffic volume can incur different transfer latency depending on the relative locations of source and destination.

Naive partitioning policies, such as assigning each peer to a fixed NPU-core group, ignore both traffic skew and topology-dependent access cost. As a result, cores assigned to heavy or slow peer transfers finish much later, while other cores become idle at the local completion barrier.

StrataCL models NPU-core partitioning as a minimum-makespan problem with a per-peer fairness constraint to avoid fan-out bursts and UB contention. For each peer p , payload B_p is split into fine-grained transfer units. The unit size S_t is a tier-specific partitioning granularity chosen to minimize policy-generation overhead while keeping residual per-core imbalance bounded. Larger units reduce the cost of computing the partitioning policy, whereas smaller units provide finer-grained load balance. For a unit of size $s \leq S_t$,

StrataCL estimates its cycle cost as

$$\tau(s) = \alpha_t + \frac{s}{\beta_t},$$

where α_t is the measured access latency of tier t , and β_t is the bandwidth. Thus, uneven traffic is captured by the number of units, while non-uniform UB access is captured by tier-dependent latency and bandwidth.

Given these units, StrataCL assigns them to NPU cores and logical issue stripes. Each stripe represents one wave of concurrent remote-memory operations, where each core issues at most one transfer unit. Let $x_{c,k,j} \in \{0, 1\}$ indicate whether unit j is issued by core c in stripe k , and let τ_j be its estimated cost. The predicted load of core c is

$$L_c = \sum_k \sum_j x_{c,k,j} \tau_j.$$

The primary objective is to minimize the predicted straggler:

$$\min_c \max L_c.$$

To avoid bursty fan-out, StrataCL further bounds the number of cores that simultaneously target the same peer. For peer p , let $\mathcal{J}_p$ denote all units targeting p . The instantaneous fan-out to p in stripe k is

$$F_{k,p} = \sum_c \sum_{j \in \mathcal{J}_p} x_{c,k,j}.$$

StrataCL enforces a tier-specific cap:

$$F_{k,p} \leq H_t, \quad \forall k, p,$$

where H_t is chosen according to the UB tier. This constraint spreads remote accesses across peers at each moment, rather than concentrating many cores on the same destination.

Since exact minimum-makespan scheduling is NP-hard, StrataCL computes an approximate NPU-core partitioning policy with a longest-processing-time-first (LPT)-style list scheduler [15]. Appendixes §B proves the NP-hardness of the partitioning problem and discusses the LPT-style approximation. The output is a per-core task list, and each NPU core simply follows the list to issue remote-memory instructions in order. Profiling results show that the relative completion-time gap between the fastest and slowest NPU cores is reduced from about 43% to within 5% after applying StrataCL’s partitioning policy (details in §9.6).

The policy-generation overhead is less than 0.5% of operator latency and can be reused across iterations for regular collectives with stable tensor shapes, as detailed in Appendixes §C. For dynamic MoE dispatch/combine, StrataCL uses a hierarchical variant that preserves token-level placement while computing the NPU-core partitioning policy at expert-window and peer-window granularity, avoiding excessive token-level analysis overhead, as detailed in Appendixes §D.

6.3 NPU-Driven SDMA Offloading

Saturating UB bandwidth with full-mesh operators is core-intensive. In the default path, data is moved through NPU cores’ MTEs, keeping the cores occupied throughout the transfer. As shown in §3.2, reaching 95% of peak bus bandwidth requires using half of the NPU cores. This creates compute-resource contention under compute-communication overlap, such as FSDP parameter prefetch [77] in LLM training and Two-micro-Batch Overlapping [14] in LLM inference. To reduce NPU-core occupation, StrataCL introduces an *NPU-driven SDMA-offloaded* path, where data transfers are offloaded to the asynchronous SDMA engine, leaving NPU cores available for concurrent computation.

The main challenge is issuing SDMA transfers with low control overhead. A conventional host-triggered path requires each transfer to pass through the CANN runtime and host control path, which is too expensive for communication operators with many fine-grained transfers. StrataCL therefore issues SDMA entirely from the device side. During execution, NPU cores construct SDMA descriptors, submit them to per-core hardware queues concurrently, and ring the SDMA doorbell directly, as shown in Figure 9a.

StrataCL supports two completion mechanisms depending on how the transferred data is consumed. For *in-kernel consumption*, such as fused dispatch or combine, synchronization remains on the device side. After normal SDMA data descriptors, the sender appends a small tail descriptor that writes a status flag to the peer rank, and a few NPU cores poll local status flags to detect completion. For *cross-kernel consumption*, such as collective communication followed by a separate compute kernel, StrataCL uses SDMA notification. A notify record is appended to the same ordered SDMA queue, and the AI CPU waits for this notification before allowing the downstream to proceed. This mode introduces a kernel-level synchronization point, but fully releases NPU cores after descriptor submission, making it suitable for throughput-oriented overlap scenarios, as shown in Figure 9b.

SDMA offloading trades a small latency penalty for much lower NPU-core occupation. It is about 9% slower than the MTE path due to descriptor-construction overhead, but reduces NPU-core occupation by over 95%. This tradeoff benefits compute-communication overlap by leaving more NPU cores available to concurrent compute kernels (§9.5).

6.4 NPU-Side Profiler

Efficient communication-operator design on CM384 requires fine-grained visibility into intra-kernel behavior, including synchronization overhead and NPU-core imbalance. Existing profilers are insufficient for this purpose. System-level profilers, such as Nsight Systems [44] and MindStudio [21], mainly provide inter-kernel timelines with coarse-grained

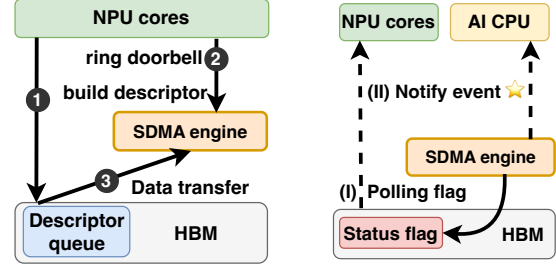


Figure 9. (a) Device-side SDMA issue path. (b) Two completion mechanisms: (I) NPU cores poll a status flag for in-kernel synchronization, and (II) the SDMA engine raises a notify event that a downstream or AI CPU waits on.

attribution. Hardware-metric profilers, such as Nsight Compute [43], expose hardware information but do not directly attribute latency to developer-defined communication stages.

To address this gap, StrataCL provides a lightweight NPU-side profiler that records per-stage timing inside communication kernels. The profiler inserts inline probes at protocol-stage boundaries and uses the on-core cycle counter to timestamp developer-defined stages. Each NPU core writes events to its own slice of a host-pinned trace buffer, and then decodes the trace into per-core timelines for inspecting per-stage latency. The profiler supports both manual and automatic instrumentation. In manual mode, developers insert named begin/end probes around semantic regions. In automatic mode, inspired by Neutrino’s programmable assembly-level GPU probing [17], StrataCL injects probes into compiled kernels without source-code modification. The collected traces expose per-core and per-stage latency, helping identify synchronization tails, topology-dependent slow paths, and excessive NPU-core usage, which in turn guides the optimization of efficient communication operators.

7 Implementation

StrataCL is implemented on top of the CANN stack [18]. The implementation contains about 21K lines of Ascend C/C++ and 5K lines of Python, and is integrated with PyTorch [51], SGLang [79], TorchTitan [34], and TorchRec [28].

Interception layer. StrataCL makes registration-on-allocation transparent by intercepting framework memory APIs. It interposes CANN allocation and mapping entry points, including `acLrtMalloc` and `acLrtMapMem`. The hook is triggered only by physical-memory events, such as newly created caching-allocator segments or VMM page mapping.

CPU-side direct DRAM access. StrataCL enables host CPUs to access peer DRAM over UB with regular load/store instructions, which is not supported by the default supernode configuration. StrataCL extends the global UB physical-address map with a remote-DRAM range, configures BIOS routing through CPU-connected UB switch planes, and installs address-decode and translation entries in UB switches.

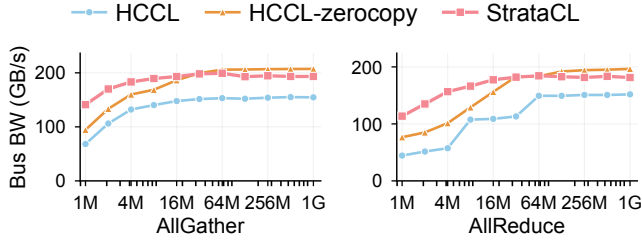


Figure 10. Bus bandwidth of AllGather and AllReduce.

Remote-DRAM UB addresses are then mapped into the accessing process’s CPU page table, allowing CPU cores to access peer host DRAM through ordinary virtual addresses. This direct path reduces metadata-broadcast latency from 60 μ s to 7–8 μ s compared with the NPU-relay path.

Generality on NVIDIA GPUs. StrataCL relies on a scale-up fabric with a global address space to support registration-on-allocation, and similar capabilities are available on NVIDIA supernodes such as GB200 NVL72 [41]. To evaluate this portability, a prototype integrates StrataCL with the CUDA runtime and NCCL on NVIDIA GPUs. The results show that StrataCL can also move user-buffer registration off the communication critical path and improve NCCL-based communication, as detailed in §9.3.

8 Evaluation

This section evaluates StrataCL at both the operator level (§8.1) and the end-to-end application level (§8.2). All experiments are conducted on a CM384 supernode.

8.1 Microbenchmark

Settings and metrics. This section evaluates two operator families at the 32-rank scale, using 32 NPU dies across two nodes. For collectives, the per-rank payload is swept from 1 MiB to 1 GiB. For MoE dispatch/combine, the evaluation follows prior settings [9, 33]: 4096 tokens per batch, hidden size 7168, top-8 experts, and EP=32, covering both high-throughput (HT) and low-latency (LL) modes. Across all experiments, performance is reported as bus bandwidth (Bus BW) in GB/s, following the standard NCCL convention [50].

Baselines. For collectives, the baselines are HCCL [19], the production collective communication library for Ascend NPUs, which incorporates state-of-the-art optimizations such as NVSHMEM-style symmetric memory [46], and HCCL-zerocopy [22], which enables user-buffer direct communication. For MoE dispatch/combine, the baselines include DeepEP v2 [9] over CX7 400 Gb/s InfiniBand RDMA, a state-of-the-art expert-parallel communication library, and the Ascend implementation of DeepEP [62] enabled by HCCL, denoted as CANN EP, together with its zerocopy variant.

Collectives. Figure 10 reports AllGather and AllReduce performance. HCCL-zerocopy improves over HCCL by $\sim 1.3\times$ on average, as user-buffer direct communication removes

Table 2. MoE dispatch/combine Bus BW (GB/s) at EP=32.

	DeepEP	CANN EP	CANN EP zerocopy	StrataCL
Dispatch (HT)	61	98	106	130
Combine (HT)	61	85	93	121
Dispatch (LL)	50	61	82	107
Combine (LL)	55	68	79	108

Table 3. End-to-end workload settings.

Workload	Model	NPU dies	Parallelism
LLM inference	DS V4 Flash	192	DP+EP
LLM training	DS V3.2 671B	512	FSDP+TP+EP
Recsys training	DLRM	128	DP+MP (TW emb.)

redundant copies between user tensors and internal communication buffers. StrataCL further outperforms HCCL-zerocopy in the small-to-medium payload regime, achieving up to $1.6\times$ higher bus bandwidth. The gains come from two optimizations. First, full-mesh execution avoids the multi-step synchronization overhead. Second, workload-balanced NPU-core partitioning mitigates core-level long tails caused by traffic skew and non-uniform memory access. For large payloads, HCCL-zerocopy can slightly outperform StrataCL, with $\sim 6\%$ higher bus bandwidth on average, because full-mesh execution introduces higher fan-out and network contention when too many remote accesses are issued concurrently. This limitation can be addressed by a workload-aware operator selection policy that switches to multi-step algorithms for large payloads. Extended results for the remaining collectives are reported in Appendices §A.

MoE dispatch/combine. Table 2 reports dispatch/combine performance at EP=32. CANN EP outperforms RDMA-based DeepEP by $1.4\times$ on average, because CM384’s UB fabric provides higher physical bandwidth than CX7 400 Gb/s InfiniBand and avoids the NIC-forwarding path used by RDMA-based expert-parallel communication. Enabling zerocopy further improves CANN EP by eliminating staging copies between user buffers and communication buffers, increasing bus bandwidth by 8.8% on average in HT mode and 25.3% in LL mode. StrataCL achieves the highest bus bandwidth in all cases, outperforming CANN EP zerocopy by 22.6%–36.7%. These gains come from StrataCL’s workload-balanced NPU-core partitioning, which accounts for expert-routing imbalance and mitigates core-level long tails.

8.2 End-to-end Performance

Workloads. This section evaluates three representative production workloads, including LLM inference, LLM training, and recommendation-system (Recsys) training (Table 3).

LLM inference: DeepSeek (DS) V4 Flash [12] is evaluated with SGLang [79] under a disaggregated serving setting [53, 80]. Each prefill instance uses DP=32 for attention layers and EP=32 for MoE layers, while each decode instance uses

EP=16 and DP=16. The serving stack further adopts state-of-the-art scheduling mechanisms, including Two-micro-Batch Overlapping (TBO) [10, 14] and Multi-Token Prediction (MTP) [13]. The deployment uses four prefill instances on eight nodes and four decode instances on four nodes, totaling 192 NPU dies. The request length distribution follows the Splitwise conversation dataset [53], whose average input-to-output length ratio is approximately 8:1.

LLM training: DeepSeek V3.2 671B [11] is evaluated with TorchTitan [34] on 512 NPU dies. The training run uses hybrid parallelism with FSDP=128, TP=4, and EP=64. The global batch size is 512, and the sequence length is 4096. This workload includes both common collectives and MoE-specific communication: FSDP introduces parameter AllGather and gradient ReduceScatter, TP invokes AllReduce, and MoE layers invoke dispatch/combine for expert routing. FSDP parameter prefetch is enabled, allowing parameter AllGather to overlap with adjacent-layer computation.

Recsys training: DLRM [39] is evaluated with TorchRec [28] on 128 NPU dies using a standard hybrid-parallel deployment. The embedding tables are table-wise sharded across dies, while the dense MLPs are replicated with data parallelism. The Criteo Terabyte dataset [8] drives embedding accesses with a batch size of 1024 and a 7 TiB embedding table. This workload involves AllToAll and AllReduce. AllToAll routes sparse features and exchanges pooled embeddings across embedding-table owners, while AllReduce synchronizes dense-MLP gradients across data-parallel replicas.

Metrics. For LLM inference, performance is measured under different request rates. The primary metric is normalized latency, computed as end-to-end request latency divided by the number of generated tokens. P99 time-to-first-token (TTFT) and P99 time-per-output-token (TPOT) are also reported. For training, iteration time is used as the primary metric.

Baselines. StrataCL is compared with two baselines, HCCL and HCCL-zerocopy. Since the native HCCL-zerocopy interface is not integrated with existing production frameworks, HCCL-zerocopy is implemented as a practical and optimistic baseline using pool-based registration. Specifically, the whole HBM is pre-registered as communication-accessible memory, and user tensors are allocated from this pool, avoiding just-in-time registration on the critical path.

LLM inference. Figure 11a reports normalized latency under different request rates. HCCL-zerocopy improves serving throughput over HCCL by only 1.2 $\times$. Although user-buffer direct communication reduces communication latency, its pre-registered memory pool disables PyTorch’s expandable-segment allocator and adds 1.6 GiB of fragmentation. This forces the serving batch size to drop by 3 to avoid out-of-memory errors, partially offsetting the operator-level benefit.

StrataCL avoids this problem by registration-on-allocation, preserving PyTorch’s expandable-segment mechanism and

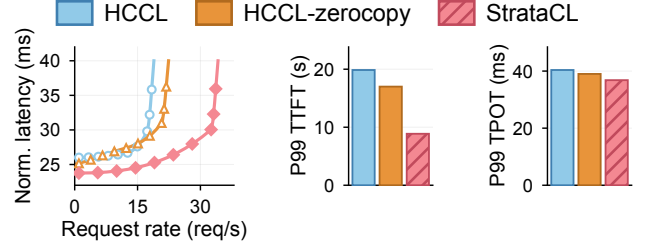


Figure 11. LLM inference performance. (a) Normalized latency vs. request rate. (b) P99 TTFT and P99 TPOT at 15 req/s.

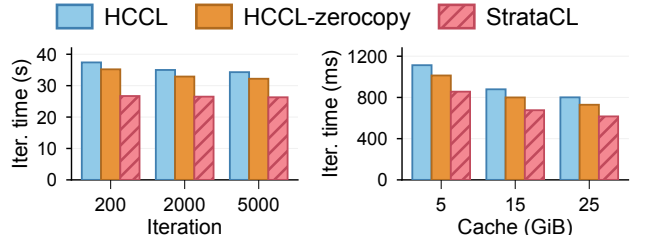


Figure 12. (a) LLM training iteration time at different training iterations. (b) Recsys training iteration time under different embedding cache sizes. Cache size here refers to the memory allocated to the embedding table for each NPU.

maintaining the same serving batch size as HCCL. As a result, StrataCL improves inference throughput by 1.9 $\times$ over HCCL and 1.6 $\times$ over HCCL-zerocopy. The gain comes from workload-balanced NPU-core partitioning and SDMA offloading, which prevent communication kernels from competing with compute kernels for NPU cores under TBO.

Figure 11b further reports P99 TTFT and P99 TPOT at 15 req/s. Compared with HCCL, StrataCL reduces P99 TTFT and P99 TPOT by 2.2 $\times$ and 1.1 $\times$, respectively. These results show that StrataCL improves both peak serving throughput and tail latency under high-load serving conditions.

LLM training. Figure 12a reports LLM training iteration time. HCCL-zerocopy provides only a modest improvement over HCCL, reducing iteration time by 6%, because training imposes stronger and longer-lived memory pressure than inference, making memory fragmentation more severe, which introduces about 3 GiB of additional fragmentation. In contrast, StrataCL reduces iteration time by 18%–24% over HCCL-zerocopy. This gain comes from registration-on-allocation, which avoids pool-based fragmentation, and SDMA offloading, which reduces NPU-core contention between FSDP prefetch communication and concurrent computation. The benefit becomes smaller in later iterations because gate layers gradually improve expert load balance [64], making traffic distribution more uniform and reducing the opportunity for workload-balanced NPU-core partitioning.

Recsys training. Figure 12b reports the Recsys training iteration time. StrataCL reduces iteration time by ~23% over

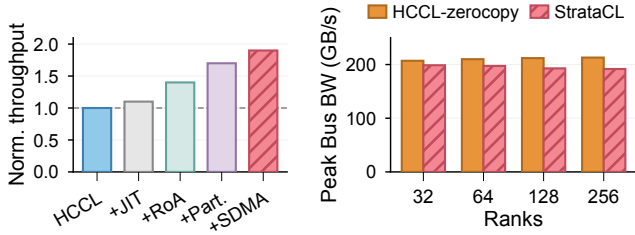


Figure 13. (a) Performance breakdown of StrataCL’s LLM inference throughput (JIT: just-in-time user-buffer registration, RoA: registration-on-allocation, Part.: workload-balanced NPU-core partitioning). (b) Peak AllGather bus bandwidth.

HCCL and $\sim 16\%$ over HCCL-zero-copy, demonstrating its effectiveness for sparse embedding communication. The communication benefit becomes more pronounced when the embedding cache is smaller, or the per-worker batch size is larger, because both cases increase embedding-cache misses and trigger more remote embedding transfers, making communication a larger fraction of iteration time.

9 Ablation Study

9.1 Performance Breakdown

Figure 13a decomposes StrataCL’s end-to-end LLM inference gain by enabling one technique at a time on top of HCCL. Registering user buffers just-in-time yields only $1.1\times$, because per-buffer registration latency consumes most of the zero-copy benefit. Registration-on-allocation alone improves throughput to $1.4\times$, because it enables user-buffer direct communication without the registration overhead on the critical path. Adding workload-balanced NPU-core partitioning raises the gain to $1.7\times$ by reducing core-level long tails in collectives and dispatch/combine operators. Finally, SDMA offloading releases NPU cores during data transfers, lifting the throughput to $1.9\times$. This is because communication no longer competes with computation for NPU cores.

9.2 Communication Operator Scalability

Figure 13b compares the peak AllGather bus bandwidth of StrataCL and HCCL-zero-copy as the communicator scales from 32 to 256 ranks. HCCL-zero-copy sustains a slightly higher peak bandwidth with its multi-step ring algorithm, while StrataCL’s full-mesh execution becomes modestly lower at larger scales due to increasing fan-out and network contention from concurrent remote accesses. However, this gap grows sublinearly with rank count and remains within 10% even at 256 ranks. The remaining large-payload peak-bandwidth gap can be further reduced by a workload-aware operator selection policy that switches to multi-step algorithms for large payloads.

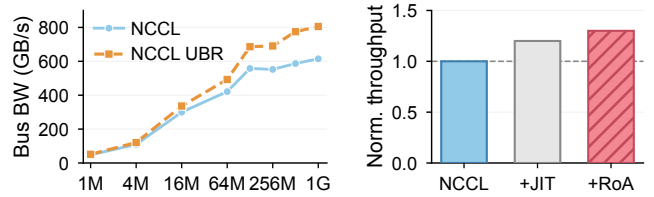


Figure 14. (a) AllGather Bus BW of NCCL vs. NCCL UBR. (b) DeepSeek V4 Flash [12] serving throughput with SGLang.

9.3 Generality on NVIDIA GPUs

To validate the generality, a prototype is implemented on NVIDIA GPUs and evaluated on an NVIDIA DGX B200 server with NVLink 5.0, serving as a smaller NVLink-domain proxy for NVIDIA supernodes such as GB200 NVL72 [41]. Figure 14a reports 8-rank AllGather performance with NCCL and NCCL User Buffer Registration (UBR) [48]. NCCL UBR removes staging copies and improves bus bandwidth over baseline NCCL across all payloads, with an average gain of $1.2\times$.

Figure 14b reports end-to-end serving throughput for DeepSeek V4 Flash [12] with SGLang [79]. NCCL UBR with just-in-time registration improves throughput to $1.2\times$ by enabling user-buffer direct communication, while NCCL with registration-on-allocation further reaches $1.3\times$ by moving buffer registration off the communication critical path. The gain is moderate in this 8-GPU setting because the cost of just-in-time registration is not strongly amplified at a small scale. At larger NVLink-domain scales, the benefit of registration-on-allocation is expected to increase because registration overhead grows with the number of peers, consistent with the scaling behavior observed on CM384. These results show that the core idea of registration-on-allocation is not specific to CM384 and can also benefit NCCL-based communication on NVIDIA scale-up fabrics.

9.4 Allocation-to-communication gap statistics

This section evaluates whether registration-on-allocation can hide remote-access registration from the communication critical path (§5.1). Across the three production workloads, StrataCL profiles each communication-operator invocation and traces the accessed buffers back to their physical allocation time using PyTorch’s caching-allocator memory history [59]. As shown in Figure 15a, even the minimum allocation-to-communication gap is several seconds, orders of magnitude larger than the microsecond- to millisecond-scale cost of remote-access registration. This interval mainly consists of model warm-up activities, such as Graph capture [68] and KV-cache [57] allocation in LLM inference or model weight loading in LLM training. These results confirm

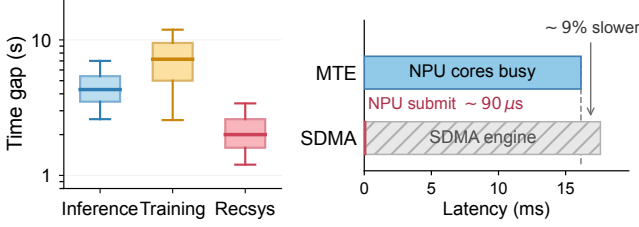


Figure 15. (a) Allocation-to-communication gap. (b) NPU-core occupation time under SDMA offloading.

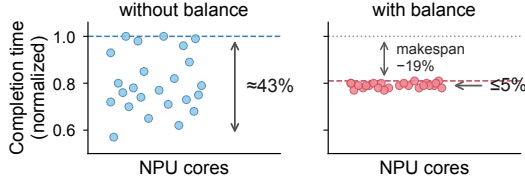


Figure 16. Per-NPU-core completion time statistics.

that registration-on-allocation can remove remote-access registration from the communication critical path.

9.5 NPU-driven SDMA Offloading

This section evaluates whether SDMA offloading reduces NPU-core occupation (§6.3). Using the NPU-side profiler, a 32-rank 128 MiB AllGather is profiled under two paths. In the MTE path, NPU cores directly perform data movement. In the SDMA-offloaded path, NPU cores only submit descriptors, while the SDMA engine transfers data asynchronously.

As shown in Figure 15b, the MTE path keeps NPU cores busy for almost the entire transfer, while the SDMA-offloaded path uses them only for descriptor submission, reducing NPU-core occupation by over 95%. This comes with a modest 9% latency slowdown due to descriptor construction and doorbell submission. This tradeoff is favorable because released NPU cores reduce contention with concurrent compute kernels and improve end-to-end throughput.

9.6 NPU-Core Workload Balance

This section evaluates whether workload-balanced NPU-core partitioning reduces core-level long tails (§6.2). Using the NPU-side profiler, per-core completion time is collected on a 32-rank 128 MiB AllGather under naive equal-core partitioning and StrataCL’s workload-balanced partitioning.

As shown in Figure 16, naive partitioning creates a clear tail because cores assigned to slower or heavier peer transfers finish much later, leaving faster cores idle. The fastest-to-slowest completion-time gap reaches about 43%. By assigning transfer units according to modeled tier cost and bounding instantaneous fan-out, StrataCL reduces the gap to within 5% and lowers the makespan by 19%.

10 Related Work

Collective communication libraries. Vendor stacks such as NCCL, HCCL, and RCCL provide collectives for GPU and NPU [4, 19, 42], while NVSHMEM, rocSHMEM, and CANN SHMEM expose PGAS-style remote memory operations to device kernels [5, 20, 46, 69]. Programmable libraries such as MSCCL++ and MSCCLang provide algorithm- or schedule-level primitives for customized optimization [7, 23], and TACCL synthesizes topology-aware collective algorithms from communication sketches [63]. NCCL UBR [48] and HCCL-zero-copy [22] reduce redundant copies through user-buffer direct communication, but still rely on just-in-time registration. StrataCL advances beyond these mechanisms with registration-on-allocation, which registers physical allocations asynchronously, remains compatible with VMM API-based allocators, and keeps user-buffer direct communication transparent to production frameworks.

Expert-parallel communication. MoE models rely on expert-parallel dispatch and combine, which have become major communication bottlenecks in large-scale training and serving. DeepEP, Tutel, FlashMoE, and MegaScale-MoE optimize token shuffling, fused dispatch/combine kernels, expert routing, and communication-computation overlap [2, 9, 24, 31]. SwiftEP further reduces staging overhead through buffer fusion [33]. These techniques mainly optimize how MoE data movement is packed, scheduled, fused, or overlapped, while remote-accessible buffers are still typically prepared through pre-allocated memory pools. StrataCL instead removes registration from the communication critical path through registration-on-allocation and optimizes dispatch/combine considering supernode architectural features.

Communication in distributed ML systems. Communication efficiency is a first-class concern across the distributed-ML stack. Alibaba HPN tailors datacenter networks to LLM traffic [61], Vela provides virtualized GPU-direct RoCE fabrics [38], and HostNet redesigns host networking by separating a zero-copy data path from a flexible control path [66]. MegaScale co-designs parallelism and communication to scale training beyond 10,000 GPUs [30], Alpa automatically derives parallelization plans [78], and TopoOpt co-optimizes network topology and parallelization strategy [70]. What-if analysis diagnoses communication stragglers [37], while PipeMorph adapts pipeline schedules to tolerate communication jitter [73]. These systems optimize at the cluster and framework layers, which are orthogonal to StrataCL’s focus on communication-library optimization.

Communication-computation overlap. Another line of work hides communication latency behind computation. T3 transparently tracks and triggers collectives to overlap them with dependent computation [55], while TokenWeave splits token batches so that one wave’s communication overlaps

with another wave’s computation [14]. Lagom co-tunes communication and computation to maximize overlap in distributed LLM training [75]. Recent systems also offload communication to dedicated engines. ARK lets GPU kernels drive DMA engines without CPU intervention [25], while ConCCL and DMA Collectives offload concurrent collectives to GPU DMA/copy engines [1, 56]. StrataCL achieves a similar goal in the supernode by integrating NPU-driven SDMA offloading. This releases NPU cores and avoids compute-resource contention in production overlap scenarios.

11 Conclusion

This paper presents StrataCL, a zero-redundancy and fabric-native communication library for production supernodes. Through the registration-on-allocation mechanism, StrataCL removes staging copies and keeps remote-access registration off the critical path while remaining transparent to applications. It further reduces synchronization overhead, core-level long tails, and core contention through full-mesh execution, workload-balanced NPU-core partitioning, and SDMA offloading. Evaluation on CM384 shows that StrataCL improves collective and MoE communication and delivers significant end-to-end gains across LLM inference, LLM training, and Recsys training, demonstrating its practicality for accelerating distributed AI applications on production supernodes.

References

- [1] Anirudha Agrawal, Shaizeen Aga, Suchita Pati, and Mahzabeen Islam. 2025. ConCCL: Optimizing ML Concurrent Computation and Communication with GPU DMA Engines. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS 25)*. 1–11. doi:10.1109/ISPASS64960.2025.00018
- [2] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. 2025. FlashMoE: Fast Distributed MoE in a Single Kernel. In *Advances in Neural Information Processing Systems (NeurIPS 25)*. <https://arxiv.org/abs/2506.04667>
- [3] AMD. 2024. AMD Instinct MI300X Accelerator Data Sheet. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-data-sheet.pdf>.
- [4] AMD. 2024. RCCL: ROCm Communication Collectives Library. <https://github.com/ROCm/rccl>.
- [5] AMD ROCm. 2025. rocSHMEM: GPU-Centric Intra-Kernel Networking through an OpenSHMEM-like Interface. <https://github.com/ROCm/rocSHMEM>.
- [6] Quentin Anthony, Benjamin Michalowicz, Jacob Hatef, Lang Xu, Mustafa Abdul Jabbar, Aamir Shafi, Hari Subramoni, and Dhaleswar K. Panda. 2024. Demystifying the Communication Characteristics for Distributed Transformer Models. In *Proceedings of the 31st IEEE Symposium on High-Performance Interconnects (HOTI 24)*. <https://doi.org/10.1109/HOTI63208.2024.00020>
- [7] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 23)*. <https://doi.org/10.1145/3575693.3575724>
- [8] Criteo AI Lab. 2015. Criteo 1TB Click Logs dataset. <https://ailab.criteo.com/criteo-1tb-click-logs-dataset/>.
- [9] DeepSeek-AI. 2025. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>.
- [10] DeepSeek-AI. 2025. deepseek-ai/profile-data: Analyze computation-communication overlap in V3/R1. <https://github.com/deepseek-ai/profile-data>. Accessed: 2026-05-28.
- [11] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. arXiv:2512.02556 [cs.CL] <https://arxiv.org/abs/2512.02556>
- [12] DeepSeek-AI. 2026. DeepSeek-V4 Flash: Towards Highly Efficient Million-Token Context Intelligence. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash>. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash>
- [13] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & Faster Large Language Models via Multi-token Prediction. *arXiv preprint arXiv:2404.19737* (2024). <https://arxiv.org/abs/2404.19737>
- [14] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2025. Token-Weave: Efficient Compute-Communication Overlap for Distributed LLM Inference. *arXiv preprint arXiv:2505.11329*. <https://arxiv.org/abs/2505.11329>
- [15] R. L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM J. Appl. Math.* (1969). <https://doi.org/10.1137/0117039>
- [16] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiyang Zhang. 2022. Clio: A Hardware-Software Co-Designed Disaggregated Memory System. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 22)*. <https://doi.org/10.1145/3503222.3507762>
- [17] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. <https://www.usenix.org/conference/osdi25/presentation/huang-songlin>
- [18] Huawei. 2024. CANN: Compute Architecture for Neural Networks. <https://www.hiascend.com/en/cann>. Huawei Ascend heterogeneous computing architecture. Accessed 2026-06-02.
- [19] Huawei. 2024. CANN-HCCL: Huawei Collective Communication Library (Open-Source). <https://gitee.com/ascend/cann-hccl>.
- [20] Huawei. 2025. Ascend SHMEM: Shared-Memory Communication Library for Ascend NPUs. <https://gitee.com/ascend/shmem>.
- [21] Huawei. 2025. MindStudio. <https://www.hiascend.com/en/developer/software/mindstudio>. Huawei Ascend development toolchain. Accessed 2026-06-02.
- [22] Huawei Technologies. 2025. HCCL Zero-Copy User-Buffer Direct Communication API. https://www.hiascend.com/document/detail/zh/canncommercial/81RC1/apiref/hcclapiref/hcclcpp_07_0053.html. CANN Commercial Edition 8.1.RC1 HCCL API (C).
- [23] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, et al. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 26)*. <https://doi.org/10.1145/3779212.3790188>
- [24] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of the Sixth Conference on Machine Learning and Systems (MLSys 23)*. https://proceedings.mlsys.org/paper_files/paper/2023/file/5616d34cf8ff73942cfd5aa922842556-Paper-mlsys2023.pdf
- [25] Changho Hwang, Kyoungsoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. 2023. ARK: GPU-driven Code Execution for Distributed Deep Learning. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. <https://www.usenix.org/conference/nsdi23/presentation/hwang>

- [26] InfiniBand Trade Association. 2014. Supplement to InfiniBand Architecture Specification Volume 1 Release 1.2.1, Annex A17: RoCEv2. <https://www.infinibandta.org/ibta-announces-new-roce-specification/>.
- [27] InfiniBand Trade Association. 2023. InfiniBand Architecture Specification Volume 1, Release 1.7. <https://www.infinibandta.org/ibta-specification/>.
- [28] Dmytro Ivchenko et al. 2022. TorchRec: a PyTorch Domain Library for Recommendation Systems. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys 22)*. <https://doi.org/10.1145/3523227.3547387>
- [29] Sylvain Jeaugey, Giuseppe Congiu, Thomas Gillis, Ben Williams, and Fred Oh. 2025. New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/new-scaling-algorithm-and-initialization-with-nvidia-collective-communications-library-2-23/>
- [30] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>
- [31] Chao Jin, Ziheng Jiang, Zhihao Bai, Zheng Zhong, Juncui Liu, Xiang Li, Ningxin Zheng, Xi Wang, Cong Xie, Qi Huang, et al. 2026. MegaScale-MoE: Large-Scale Communication-Efficient Training of Mixture-of-Experts Models in Production. In *Proceedings of the 21st European Conference on Computer Systems (EUROSYS 26)*. <https://doi.org/10.1145/3767295.3769325>
- [32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 23)*. ACM, 611–626. doi:10.1145/3600006.3613165
- [33] Xingyi Li, Yadong Liu, Xiaojie Huang, Yiran Zhang, Shuai Wang, Shangguang Wang, Zhehao Lin, Yinben Xia, Chang Yu, Qihang Liu, et al. 2026. SwiftEP: Accelerating MoE Inference with Buffer Fusion and TMA Offloading. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1073–1089. <https://www.usenix.org/conference/nsdi26/presentation/li-xingyi>
- [34] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. 2025. TorchTitan: One-stop PyTorch native solution for production ready LLM pretraining. In *The Thirteenth International Conference on Learning Representations (ICLR 25)*. <https://arxiv.org/abs/2410.06511>
- [35] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, et al. 2025. UB-Mesh: A Hierarchically Localized nD-FullMesh Datacenter Network Architecture. arXiv preprint arXiv:2503.20377. <https://arxiv.org/abs/2503.20377>
- [36] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. 2021. Ascend: a Scalable and Unified Architecture for Ubiquitous Deep Neural Network Computing: Industry Track Paper. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA 21)*. <https://doi.org/10.1109/HPCA51647.2021.00071>
- [37] Jinkun Lin, Ziheng Jiang, Zuquan Song, Sida Zhao, Menghan Yu, Zhanghan Wang, Chenyuan Wang, Zuocheng Shi, Xiang Shi, Wei Jia, et al. 2025. Understanding Stragglers in Large Model Training Using What-if Analysis. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. <https://www.usenix.org/conference/osdi25/presentation/lin-jinkun>
- [38] Apoorve Mohan, Robert Walkup, Bengi Karacali, Ming-Hung Chen, Abdullah Kayi, Liran Schour, Shweta Salaria, Sophia Wen, I-Hsin Chung, Abdul Alim, et al. 2025. Vela: A Virtualized LLM Training System with GPU Direct RoCE. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 25)*. <https://doi.org/10.1145/3676641.3716280>
- [39] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, and others. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. <https://arxiv.org/abs/1906.00091>
- [40] NVIDIA. 2022. NVIDIA H100 Tensor Core GPU Architecture Whitepaper. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>.
- [41] NVIDIA. 2024. GB200 NVL72. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>.
- [42] NVIDIA. 2024. NCCL: Optimized Primitives for Collective Multi-GPU Communication. <https://github.com/NVIDIA/nccl>.
- [43] NVIDIA. 2025. Nsight Compute. <https://developer.nvidia.com/nsight-compute>. NVIDIA developer tools. Accessed 2026-06-02.
- [44] NVIDIA. 2025. Nsight Systems. <https://developer.nvidia.com/nsight-systems>. NVIDIA developer tools. Accessed 2026-06-02.
- [45] NVIDIA. 2025. NVLink & NVLink Switch: Fastest HPC Data Center Platform. <https://www.nvidia.com/en-us/data-center/nvlink/>.
- [46] NVIDIA. 2025. NVSHMEM: A Parallel Programming Interface Based on OpenSHMEM for NVIDIA GPU Clusters. <https://developer.nvidia.com/nvshmem>.
- [47] NVIDIA. 2025. Optimizing for Low-Latency Communication in Inference Workloads with JAX and XLA. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/optimizing-for-low-latency-communication-in-inference-workloads-with-jax-and-xla/>
- [48] NVIDIA. 2025. User Buffer Registration — NCCL Documentation. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/bufferreg.html>.
- [49] NVIDIA. 2026. GPUDirect RDMA (CUDA Documentation). <https://docs.nvidia.com/cuda/gpudirect-rdma/>.
- [50] NVIDIA Corporation. 2026. Performance reported by NCCL tests. <https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md>. Accessed: 2026-05-22.
- [51] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 19)*. <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [52] Pitch Patarasuk and Xin Yuan. 2009. Bandwidth optimal all-reduce algorithms for clusters of workstations. *J. Parallel and Distrib. Comput.* (2009). <https://doi.org/10.1016/j.jpdc.2008.09.002>
- [53] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA 24)*. <https://doi.org/10.1109/ISCA59077.2024.00019>
- [54] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2023. Tale of Two Cs: Computation vs. Communication Scaling for Future Transformers on Future Hardware. In *IEEE International Symposium on Workload Characterization (IISWC 23)*. <https://doi.org/10.1109/IISWC59245.2023.00026>
- [55] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 24)*. <https://doi.org/10.1145/3620665.3640410>
- [56] Suchita Pati, Mahzabeen Islam, Shaizeen Aga, and Mohamed Assem Ibrahim. 2025. DMA Collectives for Efficient ML Communication Offloads. arXiv preprint arXiv:2511.06605 (2025). <https://arxiv.org/abs/>

- 2511.06605
- [57] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of Machine Learning and Systems 5 (MLSys 23)*. <https://arxiv.org/abs/2211.05102>
 - [58] PyTorch Contributors. 2024. CUDA Semantics — PyTorch CUDA Caching Memory Allocator (Memory Management). <https://docs.pytorch.org/docs/stable/notes/cuda.html#memory-management>.
 - [59] PyTorch Contributors. 2024. Understanding CUDA Memory Usage. https://docs.pytorch.org/docs/stable/torch_cuda_memory.html. PyTorch documentation; CUDA memory snapshots via `torch.cuda.memory._record_memory_history` and `_snapshot`. Accessed 2026-06-01.
 - [60] PyTorch Team. 2024. CUDA Semantics: Expandable Segments. <https://docs.pytorch.org/docs/stable/notes/cuda.html>.
 - [61] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference (SIGCOMM 24)*. <https://doi.org/10.1145/3651890.3672265>
 - [62] SGLang Team. 2025. DeepEP-Ascend: Ascend Implementation of DeepEP. https://github.com/sgl-project/sgl-kernel-npu/tree/main/python/deep_ep.
 - [63] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association. <https://www.usenix.org/conference/nsdi23/presentation/shah>
 - [64] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR 17)*. <https://openreview.net/forum?id=B1ckMDqlg>
 - [65] Min Si, Pavan Balaji, Yongzhou Chen, et al. 2025. Collective Communication for 100k+ GPUs. arXiv preprint arXiv:2510.20171. <https://arxiv.org/abs/2510.20171>
 - [66] Athinagoras Skiadopoulos, Zhiqiang Xie, Mark Zhao, Qizhe Cai, Saksham Agarwal, Jacob Adelmann, David Ahern, Carlo Contavalli, Michael Goldflam, Vitaly Mayatskikh, et al. 2024. High-throughput and Flexible Host Networking for Accelerated Computing. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. <https://www.usenix.org/conference/osdi24/presentation/skiadopoulos>
 - [67] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of Collective Communication Operations in MPICH. *International Journal of High Performance Computing Applications* (2005). <https://dl.acm.org/doi/10.1177/1094342005051521>
 - [68] vLLM-Ascend Team. 2025. ACL Graph. vLLM-Ascend documentation, Developer Guide. https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/feature_guide/ACL_Graph.html Accessed 2026-06-09. Describes ACL Graph capture and replay for graph mode on Ascend..
 - [69] Mattias De Wael, Stefan Marr, Bruno De Fraine, Tom Van Cutsem, and Wolfgang De Meuter. 2015. Partitioned Global Address Space Languages. *Comput. Surveys* (2015). <https://doi.org/10.1145/2716320>
 - [70] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. TopoOpt: Co-optimizing Network Topology and Parallelization Strategy for Distributed Training Jobs. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. <https://www.usenix.org/conference/nsdi23/presentation/wang-weiyang>
 - [71] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, et al. 2023. SRNIC: A Scalable Architecture for RDMA NICs. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. <https://www.usenix.org/conference/nsdi23/presentation/wang-zilong>
 - [72] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhan Yang, Rong Chen, and Haibo Chen. 2023. No Provisioned Concurrency: Fast RDMA-codedesigned Remote Fork for Serverless Computing. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. <https://www.usenix.org/conference/osdi23/presentation/wei-rdma>
 - [73] Tianyuan Wu, Lunxi Cao, Hanfeng Lu, Xiaoxiao Jiang, Yinghao Yu, Siran Yang, Guodong Yang, Jiamang Wang, Lin Qu, Liping Zhang, et al. 2026. Attack of the Bubbles: Straggler-Resilient Pipeline Parallelism for Large Model Training. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. <https://www.usenix.org/conference/nsdi26/presentation/wu-tianyuan>
 - [74] Ao Xiao, Bangzheng He, Baoquan Zhang, Baoxing Huai, Bingji Wang, et al. 2025. xDeepServe: Model-as-a-Service on Huawei CloudMatrix384. arXiv preprint arXiv:2508.02520. <https://arxiv.org/abs/2508.02520>
 - [75] Guanbin Xu, ZhenGuo Xu, Yuzhe Li, Youhui Bai, Ping Gong, Chaoyi Ruan, and Cheng Li. 2026. Lagom: Unleashing the Power of Communication and Computation Overlapping for Distributed LLM Training. arXiv preprint arXiv:2602.20656. <https://arxiv.org/abs/2602.20656>
 - [76] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. <https://www.usenix.org/conference/osdi22/presentation/you>
 - [77] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proc. VLDB Endow.* 16, 12 (2023), 3848–3860. <https://arxiv.org/abs/2304.11277>
 - [78] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. <https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>
 - [79] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS 24)*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
 - [80] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
 - [81] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems 35 (NeurIPS 22)*. https://proceedings.neurips.cc/paper_files/paper/2022/hash/2f00ecd787b432c1d36f3de9800728eb-Abstract-Conference.html
 - [82] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. 2025. MegaScale-Infer: Efficient Mixture-of-Experts

Model Serving with Disaggregated Expert Parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM 25)*. <https://doi.org/10.1145/3718958.3750506>

- [83] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, et al. 2025. Serving Large Language Models on Huawei CloudMatrix384. arXiv preprint arXiv:2506.12708. <https://arxiv.org/abs/2506.12708>

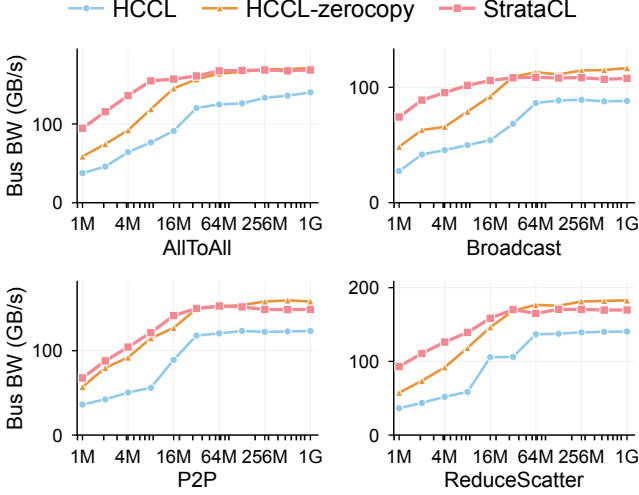


Figure 17. Extended collective microbenchmarks.

APPENDICES

A Extended Microbenchmark

Figure 17 reports the remaining four collective primitives, namely AllToAll, Broadcast, P2P, and ReduceScatter, under the same setup as §8.1. The results show a consistent trend with AllGather and AllReduce. HCCL-zero copy improves over HCCL by eliminating redundant staging copies, while StrataCL further improves performance in the small-to-medium payload regime. For P2P transfer, StrataCL achieves performance close to HCCL-zero copy, because both paths reduce to a single direct peer-to-peer data transfer with little algorithmic synchronization or workload-partitioning opportunity.

B Complexity of NPU-Core Partitioning

This appendix analyzes the complexity of the NPU-core partitioning problem in §6.2. Given transfer units $\mathcal{J}$, C NPU cores, estimated unit costs τ_j , peer mapping $p(j)$, and tier-specific fan-out caps H_t , the decision problem asks whether the units can be assigned to cores and issue stripes such that the predicted makespan is at most T while satisfying the per-peer fan-out cap:

$$\max_c \sum_k \sum_j x_{c,k,j} \tau_j \leq T, \quad \sum_c \sum_{j \in \mathcal{J}_p} x_{c,k,j} \leq H_{t(p)}, \quad \forall k, p,$$

where $x_{c,k,j} \in \{0, 1\}$ indicates that unit j is issued by core c in stripe k , each unit is assigned exactly once, each core issues at most one unit per stripe, and $\mathcal{J}_p = \{j \mid p(j) = p\}$ denotes the units targeting peer p , whose UB tier is $t(p)$.

NP-hardness by reduction. The decision version of NPU-core partitioning is NP-hard by reduction from the classical minimum-makespan scheduling problem on identical machines, $P||C_{\max}$. Given an instance of $P||C_{\max}$ with C machines, jobs $\mathcal{J}$, processing times a_j , and target makespan T , construct an NPU-core partitioning instance with C cores

and one transfer unit per job. Set each unit cost to $\tau_j = a_j$, place all units in the same UB tier and peer group, and set the fan-out cap to $H_t = C$. Since each core issues at most one transfer unit per stripe, any stripe contains at most C units in total, so the fan-out cap is never binding. The problem then reduces exactly to assigning jobs to C cores while minimizing the maximum per-core load. Thus, the constructed NPU-core partitioning instance has a feasible solution with makespan at most T if and only if the original $P||C_{\max}$ instance has a feasible schedule with makespan at most T . Since $P||C_{\max}$ is NP-hard, NPU-core partitioning is also NP-hard.

LPT-style approximate partitioning. Because exact solving requires expensive combinatorial optimization, StrataCL uses a lightweight longest-processing-time-first (LPT)-style list scheduler [15]. Without the fan-out constraint, this reduces to classical LPT scheduling, whose makespan satisfies

$$C_{\max}^{\text{LPT}} \leq \left(\frac{4}{3} - \frac{1}{3C} \right) C_{\max}^*,$$

where $C_{\max}^*$ is the optimal makespan. In StrataCL, the stripe-level fan-out cap makes the problem a constrained variant, so the classical bound does not directly apply.

C Workload Partitioning Overhead Analysis

This appendix analyzes the overhead of computing the workload partitioning policy used in §6.2. Let $U = \sum_p [B_p / S_{t(p)}]$ be the total number of atomic transfer units, and let C be the number of NPU cores used for communication. To generate the partitioning policy, StrataCL first sorts transfer units by their estimated cycle cost and then assigns each unit to the least-loaded feasible NPU core under the stripe-level fan-out cap. Therefore, the theoretical time complexity is

$$O(U \log U + U \log C),$$

where $O(U \log U)$ comes from sorting transfer units and $O(U \log C)$ comes from maintaining the least-loaded core during LPT-style list assignment.

In practice, this overhead is small. For the 32-rank 128 MiB AllGather used in the evaluation, each rank computes the partitioning policy over 24 communication NPU cores. Generating the policy takes about 40 μs , which is less than 0.5% of the operator end-to-end latency. This cost can also overlap with metadata exchange, further reducing its exposure on the critical path. Moreover, the policy is reusable: for regular collectives with stable tensor shapes, StrataCL caches the workload partitioning policy, so later invocations reuse the cached per-core task queues without recomputation.

D Hierarchical Workload Partitioning for MoE Dispatch/Combine

MoE dispatch and combine have dynamic communication patterns because expert routing changes across batches. A

naive design could treat every routed token as an independent unit when computing the workload partitioning policy. This would make the policy-generation overhead scale with the number of routed tokens, which is especially expensive during prefill when the token count can be large.

StrataCL avoids this overhead with a hierarchical workload partitioning design. The first level is *token placement*. For each batch, the MoE routing path already computes the destination expert, per-expert counts, prefix-sum offsets, and token-local offsets required by dispatch and combine. StrataCL reuses this metadata and does not introduce an additional token-level optimization pass. The second level is *expert-window aggregation*. Tokens with the same destination peer and expert are grouped into a contiguous expert window, identified by a base offset and token-local indices. This preserves token-level placement while exposing compact communication ranges. The third level is *peer-window*

partitioning. StrataCL aggregates expert windows by peer to obtain the peer payload B_p , UB tier $t(p)$, and transfer-unit count $n_p = \lceil B_p / S_{t(p)} \rceil$. The NPU-core partitioning policy is then computed over peer-window ranges under the same cycle-cost model and stripe-level fan-out cap, instead of materializing every routed token as a separate scheduling unit.

This hierarchy separates token placement from NPU-core workload partitioning. Token placement remains token-level for correctness, while workload partitioning is performed at expert-window and peer-window granularity. As a result, the input size for policy generation is bounded by the number of active expert windows and peers, rather than the number of routed tokens. This keeps the overhead small even for long-prefill MoE batches, while still adapting to dynamic routing skew.